

UniVerse

A Campus Companion

Dart & Flutter Concepts

Viva / Defense Reference with Real Code from the App

Department of CSE - Course CSE-3240 (Project I)

Team Sherlocked - Leading University, Sylhet

Covers: final, const, override, Future<void>, super.key, inheritance, polymorphism, static, try/catch/finally, error handling, main.dart initialization, and the complete navigation system.

PART 1 - Dart / Flutter Concept Questions

Each concept below gives a short definition, a real code reference taken directly from the UniVerse codebase (with the file it lives in), and a plain explanation of how it is used in the app.

1. final

A 'final' variable can be assigned only ONCE. After that first assignment its value cannot change. It is assigned at RUNTIME, so its value can be computed while the app is running (e.g. a value that comes from the database or Supabase.instance.client). 'final' freezes the variable (the reference), not necessarily the object's internal fields.

Code reference - lib/features/teacher/services/teacher_service.dart

```
class TeacherService {
  // Assigned ONCE at runtime - the value is fetched from the
  // running Supabase instance, so it cannot be 'const'.
  final SupabaseClient _supabase = Supabase.instance.client;
  final NotificationService _notifications = NotificationService();
}
```

Here _supabase and _notifications are created when a TeacherService object is built. They never get reassigned, so 'final' both documents that intent and lets the compiler enforce it. They cannot be 'const' because the values are produced at runtime, not known at compile time.

2. const

A 'const' value is a COMPILE-TIME constant - it must be fully known before the app runs. 'const' is stronger than 'final': the object and everything inside it is deeply frozen, and Dart 'canonicalizes' it (identical const values share a single object in memory, which is faster and saves memory). Widgets marked 'const' are skipped during rebuilds, which is a real performance win in Flutter.

Code reference - lib/core/router/route_names.dart

```
abstract class RouteNames {
  // Known at compile time -> declared const. These strings never
  // change, so the compiler can lock and reuse them everywhere.
  static const String splash = '/';
  static const String studentDashboard = '/student/dashboard';
  static const String findTeacher = '/find-teacher';
}
```

Code reference - const widget constructors (used app-wide)

```
// lib/core/router/app_router.dart
GoRoute(path: RouteNames.rooms, builder: (c, s) => const RoomsScreen()),
GoRoute(path: RouteNames.manageUsers,
  builder: (c, s) => const ManageUsersScreen()),
```

Because RoomsScreen takes no changing inputs, it is built as 'const'. Flutter reuses the same widget instance and skips rebuilding it, improving performance.

3. const vs final + "what if I use final instead of const?"

KEY DIFFERENCE: const is fixed at COMPILE TIME (before the program runs); final is fixed at RUNTIME (once, when the line executes).

- const -> value must be known when the code is compiled; deeply immutable; canonicalized (shared in memory); usable in places that REQUIRE a constant.
- final -> value is computed while running, assigned exactly once; the reference can't change, but the object's

internals still can.

Worked example

```
const String appName = 'UniVerse';      // OK: literal known at compile time
final DateTime now = DateTime.now();    // OK: value only known at RUNTIME

// const DateTime now = DateTime.now(); // COMPILE ERROR
// DateTime.now() runs at runtime, so it can never be const.
```

The exam question: "if I write final instead of const, will it CRASH?"

No - it will NOT crash. Replacing 'const' with 'final' almost always still compiles and runs correctly. 'final' is simply less strict than 'const'. What you lose is the compile-time guarantee and the optimizations, NOT correctness.

There are only two situations to remember:

- Going from const to final (relaxing): safe. e.g. 'final splash = "/>

Bottom line: a crash is a runtime failure. const/final mistakes are caught by the COMPILER first, so the usual outcome is 'won't compile', not 'crashes while running'. Using 'final' where 'const' was possible is legal and safe - it just misses an optimization.

Why it matters in this app

In RouteNames every path is 'const' because the strings are fixed forever. If we wrote 'final' instead, the app would still run identically, but we'd lose the ability to use those values inside other 'const' expressions (like 'const authPages = [RouteNames.splash, ...]' in app_router.dart, which REQUIRES its elements to be compile-time constants).

4. @override

'@override' is an annotation that tells Dart (and the reader) that this method is intentionally REPLACING a method inherited from a parent class or interface. It is not required to compile, but it makes the compiler verify that a method with that exact name/signature really exists in the parent - catching typos.

Code reference - lib/main.dart

```
class UniVerseApp extends StatelessWidget {
  final AppRouter router;
  const UniVerseApp({super.key, required this.router});

  @override // replaces StatelessWidget.build
  Widget build(BuildContext context) {
    return MaterialApp.router(
      title: AppConstants.appName,
      theme: AppTheme.dark,
      routerConfig: router.router,
    );
  }
}
```

Every Flutter widget overrides build(). State classes override initState() and dispose(); controllers (ChangeNotifier subclasses) are used through overridden lifecycle hooks. See app_shell.dart below for initState/dispose overrides.

```
// lib/core/router/app_shell.dart
class _AppState extends State<AppShell> {
  @override
  void initState() {
    super.initState();
    widget.notificationController.startRealtime();
  }

  @override
  void dispose() {
    widget.authController.removeListener(_onAuthChanged);
    super.dispose();
  }
}
```

5. Future<void>

A 'Future' represents a value that will be available LATER - the result of an asynchronous operation (network call, database query, file read). 'Future<T>' completes with a value of type T. 'Future<void>' means the async work completes but returns NO value - you await it only for its side effect (e.g. an insert finishing), not for data coming back.

Code reference - lib/features/teacher/services/teacher_service.dart

```
// Returns nothing; we await it only to know the insert + alert finished.
Future<void> cancelClass({
  required RoutineEntry entry,
  required DateTime date,
  required String reason,
  required String userId,
}) async {
  await _supabase.from(AppConstants.tableCancellations).insert({
    'routine_id': entry.id,
    'class_date': dateKey(date),
    'cancelled_by': userId,
  });
}
```

```
});  
  await _notifications.createBroadcast(/* ... */);  
}
```

Contrast with a Future that DOES return data - e.g. the rooms controller fetches a list:

```
// lib/features/rooms/controllers/rooms_controller.dart  
Future<void> fetchRoomStatuses() async {      // void: just updates state  
  _allRoutines = await _service.fetchAllRoutines(); // Future<List<...>>  
  notifyListeners();  
}
```

'async' marks a function that contains 'await'; an async function ALWAYS returns a Future. 'await' pauses inside the function until the Future completes, without freezing the whole app.

6. super.key

In Flutter every widget can take an optional 'Key'. A Key helps Flutter tell widgets apart when the widget tree rebuilds, so it can preserve state correctly. 'super.key' is modern Dart shorthand: it forwards the 'key' argument straight up to the parent (super) class constructor (StatelessWidget / StatefulWidget) instead of writing 'Key? key' and ': super(key: key)' by hand.

Code reference - lib/shared/widgets/explore_fab_menu.dart

```
class ExploreFabMenu extends StatelessWidget {
  // 'super.key' passes the key up to StatelessWidget's constructor.
  const ExploreFabMenu({super.key});
  // ...
}
```

Code reference - widget with its own fields + super.key

```
// lib/core/router/app_shell.dart
class AppShell extends StatefulWidget {
  final AuthController authController;
  final NotificationController notificationController;
  final Widget child;

  const AppShell({
    super.key, // forwarded to StatefulWidget
    required this.authController,
    required this.notificationController,
    required this.child,
  });
}
```

Before super-parameters, the same code was: 'const AppShell({Key? key, ...}) : super(key: key);'. 'super.key' is just a cleaner way to write that. Notice these constructors are also 'const' (Concept 2) because their fields are 'final'.

7. Inheritance

Inheritance lets a class REUSE and EXTEND another class using the 'extends' keyword. The child (subclass) gets all the fields and methods of the parent (superclass) and can add its own or replace existing ones (override). It models an 'is-a' relationship.

Code reference - the whole app is built on inheritance

```
// lib/main.dart
class UniVerseApp extends StatelessWidget { ... } // is-a Widget

// lib/core/router/app_shell.dart
class AppShell extends StatefulWidget { ... } // is-a Widget
class _AppShellState extends State<AppShell> { ... } // is-a State

// lib/features/rooms/controllers/rooms_controller.dart
class RoomsController extends ChangeNotifier { ... } // is-a ChangeNotifier
```

RoomsController 'extends ChangeNotifier', so it inherits notifyListeners(), addListener(), removeListener() for free - we never wrote those; we only call them. That inherited behaviour is exactly how the UI is told to rebuild. RouteNames uses 'abstract class' (a class never meant to be instantiated, only a namespace of constants) - another piece of the same class system.

8. Polymorphism (types) + 9. method override = which polymorphism?

Polymorphism = 'many forms': the same call behaves differently depending on the actual object/type behind it. Dart (like most OOP languages) has two kinds:

(a) Compile-time / static polymorphism

- Achieved by method OVERLOADING and/or generics + operator behaviour. The compiler decides which form to use. NOTE: Dart does NOT support classic method overloading (two methods with the same name, different parameters); it uses optional/named parameters and GENERICS instead. e.g. `Future<void>` vs `Future<List<RoutineEntry>>` - the same 'Future' type takes many forms via the generic `<T>`.

(b) Run-time / dynamic polymorphism

- Achieved by method OVERRIDING with inheritance. The decision of WHICH `build()` or WHICH `toString()` runs is made at RUN TIME based on the real object type. This is the form Flutter relies on constantly.

DIRECT ANSWER: method override is RUN-TIME (dynamic) polymorphism.

When you override a method, the parent type's reference can point to any subclass, and the OVERRIDDEN version runs at run time. This is also called 'dynamic dispatch'.

Code reference - run-time polymorphism via override

```
// Flutter calls build() on a Widget reference, but the OVERRIDDEN
// build() of the real subclass runs at runtime - dynamic dispatch.

// lib/main.dart
class UniVerseApp extends StatelessWidget {
  @override Widget build(BuildContext c) => MaterialApp.router(...);
}

// lib/shared/widgets/explore_fab_menu.dart
class ExploreFabMenu extends StatelessWidget {
  @override Widget build(BuildContext c) => FloatingActionButton(...);
}
```

Both are 'a Widget', both respond to `build()`, but each produces a DIFFERENT UI - the same method call, many forms. That is run-time polymorphism in action, and it is enabled by '@override' (Concept 4) on top of inheritance (Concept 7).

Code reference - generics (compile-time polymorphism)

```
// One generic factory shape reused for any type the column holds:
// lib/core/models/routine_model.dart
factory RoutineEntry.fromMap(Map<String, dynamic> map) => RoutineEntry(
  semester: (map['semester'] as int?) ?? 0,
  isActive: (map['is_active'] as bool?) ?? true,
);
// List<RoutineEntry>, Set<String>, Future<void> - 'many forms' of
// the same container types, resolved by the compiler.
```

10. static

'static' members belong to the CLASS ITSELF, not to any single object (instance) of it. You access them through the class name without creating an object. Use static for shared constants and pure helper functions that don't depend on instance data.

Code reference - static constants (route names)

```
// lib/core/router/route_names.dart - access as RouteNames.splash
abstract class RouteNames {
  static const String splash = '/';
  static const String studentDashboard = '/student/dashboard';
}
```

Code reference - static helper method (no object needed)

```
// lib/features/teacher/services/teacher_service.dart
// Called as TeacherService.dateKey(DateTime.now()) - pure, no state.
static String dateKey(DateTime d) =>
  '${d.year.toString().padLeft(4, '0')}-${d.month.toString().padLeft(2, '0')}-${d.day.toString().padLeft(2, '0')}';
```

Code reference - static factory list (the nav menu)

```
// lib/shared/widgets/app_bottom_nav.dart
// Single source of truth for each role's tabs - belongs to the
// class, not an instance, so it is 'static'.
static List<AppNavDest> destinationsFor(String? role) {
  switch (role) {
    case AppConstants.roleAdmin: return const [ /* admin tabs */ ];
    case AppConstants.roleTeacher: return const [ /* teacher tabs */ ];
    default: return const [ /* student tabs */ ];
  }
}
```

Contrast: `_supabase` in `TeacherService` is an INSTANCE field (each service object has its own), while `dateKey()` is static (one shared function for the whole class). A static method cannot read instance fields because no object is involved when it runs.

11. try / catch / finally + error handling

This is Dart's structured way to deal with errors so the app does not crash:

- try -> wrap the risky code that might throw an error/exception.
- catch -> runs ONLY if something inside try threw; here you handle/report the error (the caught error object is usually named 'e').
- finally -> ALWAYS runs afterwards, whether or not an error was thrown. Ideal for cleanup (closing things, turning a loading flag off).

Code reference - try / typed-catch in the app

```
// lib/features/teacher/services/teacher_service.dart
try {
  await _supabase.from(AppConstants.tableCancellations).insert({...});
  await _notifications.createBroadcast(/* ... */);
} on PostgrestException catch (e) {
  // Surface the REAL DB / RLS reason instead of a generic failure.
  throw Exception(e.message);
}
```

'on PostgrestException catch (e)' catches ONLY that specific Supabase error type - this is targeted error handling: the error object 'e' carries the database's real message (e.g. a row-level-security rejection), which we re-throw so the UI can show a meaningful message.

Code reference - try/catch driving UI state safely

```
// lib/features/rooms/controllers/rooms_controller.dart
Future<void> fetchRoomStatuses() async {
  try {
    _isLoading = true; _error = null; notifyListeners();
    _allRoutines = await _service.fetchAllRoutines();
    // ... build statuses ...
    _isLoading = false; notifyListeners();
  } catch (e) {
    _error = e.toString(); // store message, show error UI
    _isLoading = false;    // never get stuck on a spinner
    notifyListeners();
  }
}
```

Note on 'finally': UniVerse currently turns off _isLoading in BOTH the success path and the catch block (as above) rather than using a 'finally' clause. The cleaner, equivalent pattern is to put that single cleanup line in 'finally' so it can never be forgotten:

```
Future<void> fetchRoomStatuses() async {
  try {
    _isLoading = true; notifyListeners();
    _allRoutines = await _service.fetchAllRoutines();
  } catch (e) {
    _error = e.toString(); // runs only on failure
  } finally {
    _isLoading = false;    // ALWAYS runs - success OR failure
    notifyListeners();
  }
}
```

CONCEPT SUMMARY (as asked): an error is raised inside 'try'; 'catch (e)' handles it; 'finally' runs no matter what. And tying back to Concept 3 - 'const is fixed at compile time, final is fixed once at run time'.

PART 2 - How main.dart Initializes Everything & Connects the App

main.dart (lib/main.dart) is the single entry point. Flutter calls main() first. It wires together every major subsystem in a strict order, then hands control to the router. Below is the real file, broken into its initialization steps.

Step 0 - Bindings

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized(); // must be first
```

ensureInitialized() prepares the Flutter engine before we call any native plugins (Firebase, Supabase). Without it, plugin setup would fail.

Step 1 - System UI theming

```
AppTheme.setSystemUI(); // from lib/core/theme/app_theme.dart
```

Connects to core/theme: sets a transparent status bar with light icons so the dark theme looks right from the very first frame.

Step 2 - Firebase + Push (mobile only)

```
if (!kIsWeb) {
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform, // firebase_options.dart
  );
  await PushService.instance.init(); // core/services/push_service.dart
}
```

Connects to firebase_options.dart (generated config) and core/services/push_service.dart. Guarded by '!kIsWeb' because the project has no web Firebase config - calling it on web would throw and kill startup.

Step 3 - Supabase backend

```
await Supabase.initialize(
  url: AppConstants.supabaseUrl, // core/constants/app_constants.dart
  anonKey: AppConstants.supabaseAnonKey,
  authOptions: const FlutterAuthClientOptions(
    authFlowType: AuthFlowType.pkce, // secure OAuth flow for mobile
  ),
);
```

Connects to core/constants/app_constants.dart for the URL + anon key. After this line, every service in features/.../services can use Supabase.instance.client (that is the 'final _supabase' field we saw in TeacherService).

Step 4 - Controllers & Router

```
final authController = AuthController(); // features/auth
final appRouter = AppRouter(authController: authController); // core/router
```

AuthController (a ChangeNotifier) holds the auth state. It is injected into AppRouter, which uses it as 'refreshListenable' so navigation reacts to login/logout automatically (see Part 3).

Step 5 - Push tap + OAuth deep-link wiring

```

if (!kIsWeb) {
  PushService.instance.onNotificationTap =
    (_) => appRouter.router.go(RouteNames.notifications);
}

AuthService().authStateChanges.listen((event) async {
  if (event.event == AuthChangeEvent.signedIn) {
    if (authController.status != AuthStatus.authenticated &&
        !authController.isLoading) {
      await authController.handleOAuthCallback();
    }
    final userId = event.session?.user.id;
    if (!kIsWeb && userId != null) {
      unawaited(PushService.instance.registerToken(userId));
    }
  } else if (event.event == AuthChangeEvent.passwordRecovery) {
    appRouter.router.go(RouteNames.resetPassword);
  }
});

```

This is the glue between three features: AuthService (auth), PushService (notifications), and the router. When Google OAuth completes, Supabase fires 'signedIn'; the listener resolves the session and saves the device's FCM token (fire-and-forget via 'unawaited', so navigation is never blocked).

Step 6 - Launch the app

```

runApp(UniVerseApp(router: appRouter));
}

class UniVerseApp extends StatelessWidget {
  final AppRouter router;
  const UniVerseApp({super.key, required this.router});
  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      title: AppConstants.appName,
      debugShowCheckedModeBanner: false,
      theme: AppTheme.dark,           // core/theme
      routerConfig: router.router,    // core/router -> all screens
    );
  }
}

```

runApp() mounts the root widget. MaterialApp.router hands navigation entirely to GoRouter via routerConfig. From here on, the router decides which screen shows.

How main.dart connects to the folders (summary)

- core/theme -> AppTheme.setSystemUI(), AppTheme.dark (look & feel).
- core/constants -> AppConstants (Supabase keys, app name, table names).
- core/services -> PushService (FCM token + tap handling).
- core/router -> AppRouter + RouteNames (the entire navigation graph).
- features/auth -> AuthController + AuthService (who is logged in).
- firebase_options.dart -> generated Firebase config for Android.
- Every features/.../services file -> uses the Supabase client that main() initialized in Step 3.

PART 3 - The Complete Navigation System

UniVerse uses GoRouter (declarative routing). There is exactly ONE router, built in `core/router/app_router.dart` and handed to `MaterialApp.router` in `main.dart`. Screens NEVER use `MaterialPageRoute` - they call `context.go(...)` / `context.push(...)` with a constant from `RouteNames`. The system has four parts:

- 1. `RouteNames` - every path as a const string (one source of truth).
- 2. `AppRouter` - the GoRouter: route table + auth redirect guard.
- 3. `AppShell` + `AppBottomNav` - the tab frame for top-level screens.
- 4. Screens - call `context.go` / `context.push` to move around.

Part A - Route names (`core/router/route_names.dart`)

```
abstract class RouteNames {
  static const String splash = '/';
  static const String login = '/login';
  static const String studentDashboard = '/student/dashboard';
  static const String teacherDashboard = '/teacher/dashboard';
  static const String adminDashboard = '/admin/dashboard';
  static const String rooms = '/rooms';
  static const String findTeacher = '/find-teacher';
  // ...all other paths...
}
```

Because these are 'const', they can be reused inside other const lists (like the redirect guard's `authPages`) and the compiler guarantees no typo'd path string.

Part B - The router + redirect guard (`core/router/app_router.dart`)

`AppRouter` wraps a `GoRouter`. 'refreshListenable: `authController`' means: every time auth state changes, `GoRouter` re-runs its redirect logic. The redirect is the security gate - it decides if the user is ALLOWED on the requested page.

```
late final GoRouter router = GoRouter(
  initialLocation: RouteNames.splash,
  refreshListenable: authController, // re-check on login/logout
  redirect: (context, state) async {
    final status = authController.status;
    final location = state.matchedLocation;

    if (status == AuthStatus.unauthenticated) {
      return authPages.contains(location) ? null : RouteNames.login;
    }
    if (status == AuthStatus.authenticated) {
      if (authPages.contains(location))
        return _dashboardForRole(authController.role);
      if (_isWrongRolePage(location, authController.role))
        return _dashboardForRole(authController.role);
    }
    return null; // null = allow the navigation
  },
  routes: [ /* see Part C + D */ ],
);
```

Returning a path = redirect there; returning null = allow. `_isWrongRolePage()` blocks a student from ever opening an `/admin` or `/teacher` URL, sending them back to their own dashboard. This is why screens can navigate freely - the guard is the safety net.

```
String _dashboardForRole(String? role) {  
  switch (role) {  
    case 'teacher': return RouteNames.teacherDashboard;  
    case 'admin':   return RouteNames.adminDashboard;  
    default:        return RouteNames.studentDashboard;  
  }  
}
```

Part C - ShellRoute: the tabbed area

Top-level tab screens are wrapped in a ShellRoute. The ShellRoute builds ONE AppShell (one Scaffold + one bottom nav) and injects each tab screen as 'child'. This is why the bottom nav stays put while tabs change.

```
ShellRoute(
  builder: (c, s, child) => AppShell(
    authController: authController,
    notificationController: notificationController,
    child: child,                // the active tab screen
  ),
  routes: [
    GoRoute(path: RouteNames.studentDashboard,
      builder: (c, s) => StudentDashboardScreen(...)),
    GoRoute(path: RouteNames.studentRoutine,
      builder: (c, s) => RoutineScreen(authController: authController)),
    GoRoute(path: RouteNames.teacherDashboard,
      builder: (c, s) => TeacherDashboardScreen(...)),
    GoRoute(path: RouteNames.adminDashboard,
      builder: (c, s) => AdminDashboardScreen(...)),
    // ...other tab routes...
  ],
)
```

Secondary screens are OUTSIDE the shell (pushed, with a back button)

```
// Top-level (not tabs) -> no bottom nav, uses back button
GoRoute(path: RouteNames.rooms,          builder: (c,s) => const RoomsScreen()),
GoRoute(path: RouteNames.findTeacher,    builder: (c,s) => const FindTeacherScreen()),
GoRoute(path: RouteNames.timetableGrid,
  builder: (c, s) => TimetableGridScreen(
    rows: (s.extra as List<RoutineEntry>?) ?? const [], // pass data via extra
  )),
```

Note 's.extra' - GoRouter's way to pass an object (here a list of routine rows) to the next screen without putting it in the URL.

Part D - AppShell + AppBottomNav (the tab frame)

AppShell (core/router/app_shell.dart) owns the single Scaffold. It reads the current location from GoRouter to (a) highlight the right tab and (b) show the Explore FAB only on the three dashboards.

```
// lib/core/router/app_shell.dart (inside build)
final location = GoRouterState.of(context).matchedLocation;
return Scaffold(
  body: widget.child,
  floatingActionButton: (location == '/student/dashboard' ||
    location == '/teacher/dashboard' ||
    location == '/admin/dashboard')
    ? const ExploreFabMenu() : null,
  bottomNavigationBar: AppBottomNav(
    role: widget.authController.role, // chooses the tab set
    currentRoute: location,           // highlights active tab
    unreadNotifCount: widget.notificationController.unreadCount,
  ),
);
```

`AppBottomNav` (`shared/widgets/app_bottom_nav.dart`) picks the tab set by role via the static `destinationsFor(role)`, then navigates on tap with `context.go()`:

```
// shared/widgets/app_bottom_nav.dart
final dests = destinationsFor(role);           // role-aware tabs
var currentIndex = dests.indexWhere((d) => d.route == currentRoute);
if (currentIndex < 0) currentIndex = 0;

return UBottomNav(
  items: [for (final d in dests) UNavItem(icon: d.icon, label: d.label)],
  currentIndex: currentIndex,
  onTap: (i) {
    final route = dests[i].route;
    if (route == currentRoute) return; // ignore tapping the current tab
    context.go(route);                // <-- the actual tab switch
  },
);
```

Part E - go vs push (when each is used)

- `context.go(route)` -> REPLACES the current location. Used for switching tabs and for role dashboards (you don't want a back-stack between tabs).
- `context.push(route)` -> PUSHES on top (adds a back button). Used for secondary screens like Rooms, Find Teacher, Admin Registration, Manage Resources.
- `Navigator.pop(context)` -> closes a pushed screen or a bottom sheet/dialog.

Real UI usage - tab/dashboard navigation with go()

```
// lib/features/dashboard/screens/student_dashboard_screen.dart
onTap: () => context.go(RouteNames.studentRoutine), // go to Routine tab
onTap: () => context.go(RouteNames.resources),      // go to Resources
onTap: () => context.go(RouteNames.profile),        // go to Profile

// lib/features/admin/screens/admin_dashboard_screen.dart
onTap: () =>
  context.go('${RouteNames.routineManagement}?tab=generate'), // query param
```

Real UI usage - secondary screens with push() + the Explore FAB

```
// lib/shared/widgets/explore_fab_menu.dart
onTap: () {
  Navigator.pop(context); // close the bottom sheet first
  context.push(RouteNames.rooms); // push Room Availability (back btn)
},
onTap: () {
  Navigator.pop(context);
  context.push(RouteNames.findTeacher);
},

// lib/features/admin/screens/admin_dashboard_screen.dart
onTap: () => context.push(RouteNames.adminRegistration),
onTap: () => context.push(RouteNames.manageResources),

// lib/features/admin/screens/generate_timetable_screen.dart
context.push(RouteNames.timetableGrid, extra: result.rows); // push + data
```

Navigation flow - end to end

- 1) App launches -> `main.dart` -> `MaterialApp.router` uses `GoRouter`, starting at `RouteNames.splash`.
- 2) `AuthController` resolves the session; because it is the router's `refreshListenable`, `GoRouter` re-runs `redirect()`.

- 3) `redirect()` routes by status: not logged in -> `/login`; logged in -> the role's dashboard; wrong-role URL -> bounced back.
- 4) Inside the app, `ShellRoute` shows `AppShell` with the role's bottom nav; tapping a tab calls `context.go(...)`.
- 5) Buttons/cards open secondary screens with `context.push(...)`; the FAB opens a bottom sheet that pushes Rooms / Find Teacher.
- 6) Back button or `Navigator.pop()` returns from pushed screens. The redirect guard keeps every transition role-safe.